Module 3

HTML Forms and Semantic HTML

Agenda

1. HTML Forms
2. Input Types
3. Labels and Form Controls
4. Select Boxes and Text Areas
5. Buttons
6. Form Attributes
7. Basic Form Validation
8. Semantic HTML Elements
9. Header, Footer, Nav, Main, Section and Article
10. Introduction to Web Accessibility
11. HTML Best Practices

© Copyright Intellipaat. All rights reserved.

HTML Forms

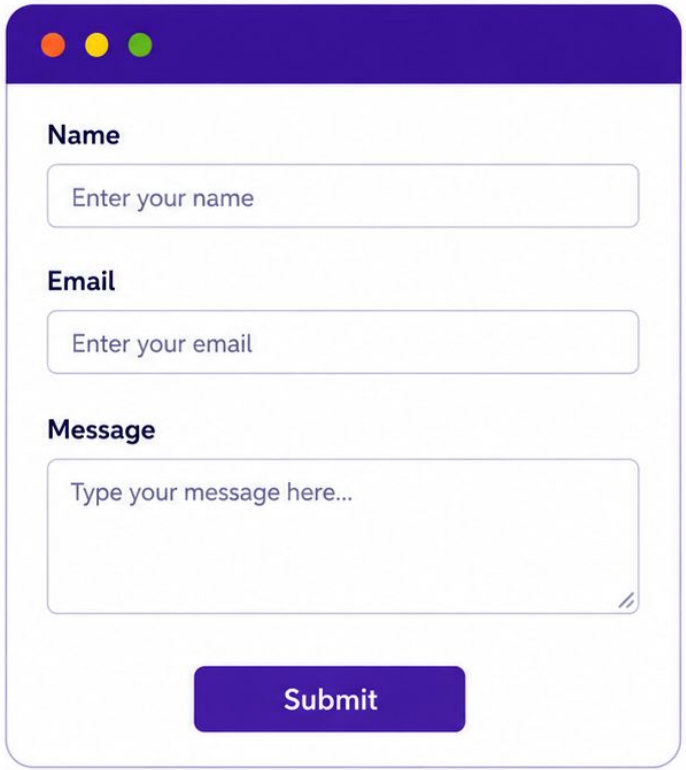
© Copyright Intellipaat. All rights reserved.

Introduction to HTML Forms

An **HTML form** is used to collect information from users and submit that information for processing.

Forms are commonly used for:

- Login and registration
- Feedback forms
- Contact forms
- Online orders
- Search boxes
- User surveys



© Copyright Intellipaat. All rights reserved.

Basics Example

The <form> element

The <form> element acts as a container for form controls such as text fields, checkboxes, radio buttons, and buttons.

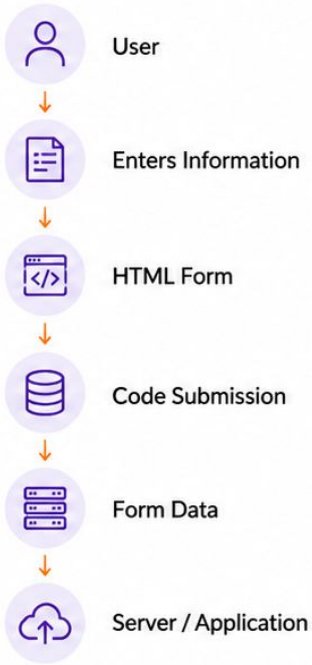
HTML Code

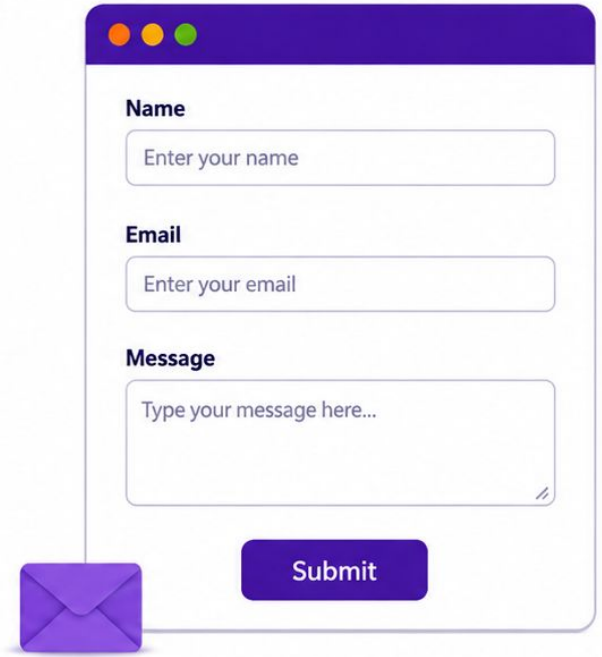
```
<form>
  <label for="name">Name:</label>
  <input type="text" id="name">

  <label for="email">Email:</label>
  <input type="email" id="email">

  <button type="submit">Submit</button>
</form>
```

How It Works





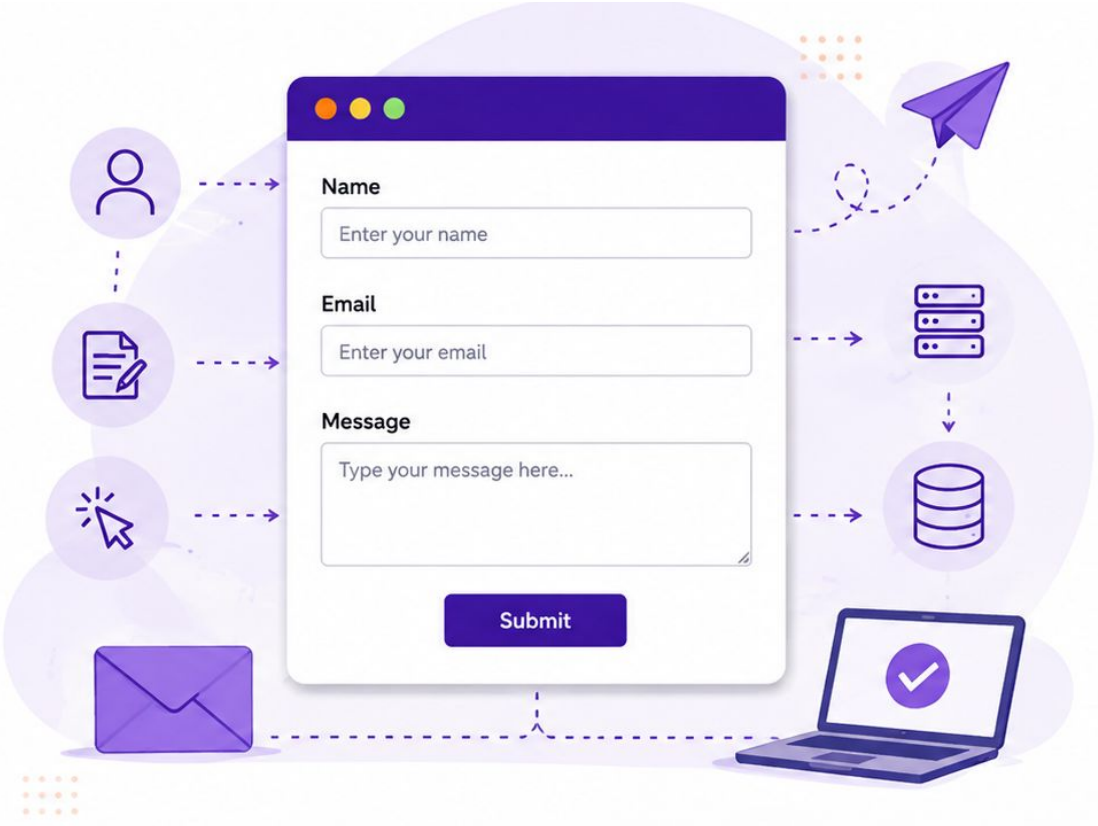
© Copyright Intellipaat. All rights reserved.

How an HTML Form Works

An HTML form follows a simple process: the **user enters information**, and the browser **submits the form data** for processing.

Form Submission Flow

- User
- Enters Form Data
- Clicks Submit
- Browser Collects Data
- Sends Form Data
- Server Processes Data
- Response Returned



© Copyright Intellipaat. All rights reserved.

Example

IntelliPaat

HTML Code

```
<form action="/register"
 method="post">

   <label for="name">Name:</label>
   <input type="text" id="name"
    name="name">

   <label for="email">Email:</label>
   <input type="email" id="email"
   name="email">

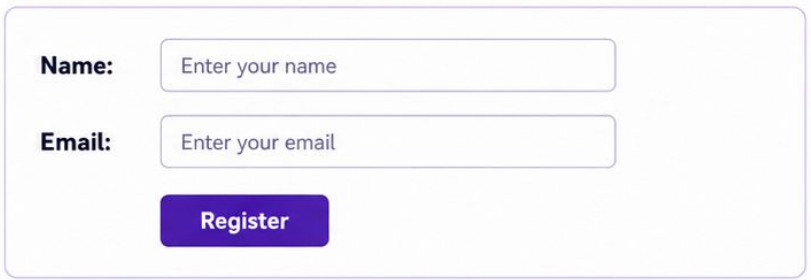
   <button type="submit">Register</button>

</form>
```

What Happens?

1. User enters their **name** and **email**.
2. User clicks **Register**.
3. The browser collects the form values.
4. The browser sends the data to **/register.**
5. The server processes the submitted information.
6. The server sends a response back to the browser.

Show Image of the output of the code



© Copyright Intellipaat. All rights reserved.

Hands-On

© Copyright Intellipaat. All rights reserved.

Hands-On: Create a Basic Registration Form

Task

Create a **Student Registration Form.**

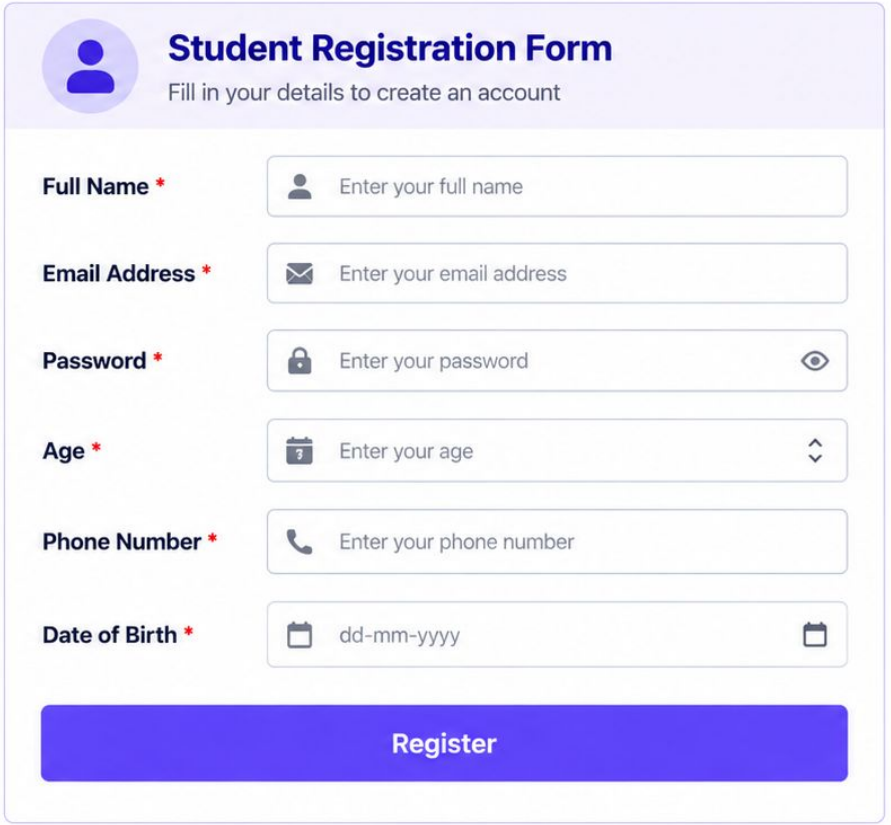
The form should collect:

- Full Name
- Email Address
- Password
- Age
- Phone Number
- Date of Birth

Include an appropriate submission control.

Expected Output

- The browser should display a registration form containing all the requested fields with their corresponding labels and a submission control.
- The student should be able to enter information into each field.



© Copyright Intellipaat. All rights reserved.

Input Types

© Copyright Intellipaat. All rights reserved.

Common HTML Input Types

The `<input>` element is used to collect different types of information from users. The `type` attribute determines what kind of input is expected.

Common Input Types

Input Type	Purpose	Example
`text`	Single-line text	Name
`password`	Hidden text	••••••••
`email`	Email address	user@example.com
`number`	Numeric value	25
`tel`	Telephone number	Phone number
`date`	Date selection	dd-mm-yyyy

© Copyright Intellipaat. All rights reserved.

Example

HTML Code

```
<form>
   <label for="name">Name:</label>
   <input type="text" id="name" name="name">

   <label for="password">Password:</label>
   <input type="password" id="password" name="password">

   <label for="email">Email:</label>
   <input type="email" id="email" name="email">

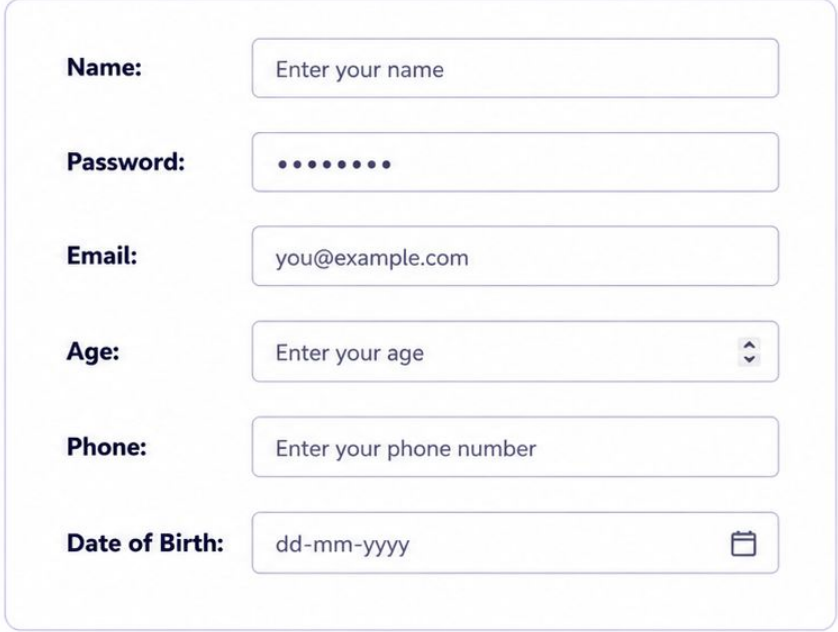
   <label for="age">Age:</label>
   <input type="number" id="age" name="age">

   <label for="phone">Phone:</label>
   <input type="tel" id="phone" name="phone">

   <label for="dob">Date of Birth:</label>
   <input type="date"  id="dob" name="dob">

</form>
```

Show Image of the output of the code



© Copyright Intellipaat. All rights reserved.

Other Useful HTML Input Types

HTML provides additional input types for collecting **choices**, **files**, **ranges**, **colors**, and **URLs**.

Common Input Types

Input Type	Purpose	Example
 `radio`	Select one option from a group	**Gender:** ◉ Male ○ Female ○ Other
 `checkbox`	Select zero or more options	**Hobbies:** ☑ Reading ☑ Music ☐ Sports
 `file`	Select a file from the device	Choose File | No file chosen
 `range`	Select a value within a range	**Volume:** ——●—— 70
 `color`	Select a color	**Theme color:** ▮ ▾
 `url`	Enter a web address	https://www.example.com

© Copyright Intellipaat. All rights reserved.

Example

```
<form>

   <p>Choose your payment method:</p>
   <input type="radio" name="payment" value="card"> Card
   <input type="radio" name="payment" value="cash"> Cash

   <p>Select your hobbies:</p>
   <input type="checkbox" name="hobby" value="music"> Music
   <input type="checkbox" name="hobby" value="sports"> Sports

   <p>Upload a profile photo:</p>
   <input type="file" name="photo">

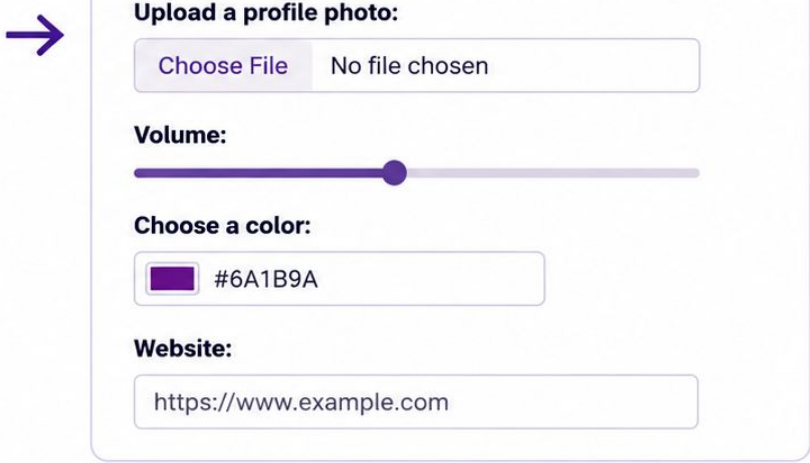
   <p>Volume:</p>
   <input type="range" name="volume">

   <p>Choose a color:</p>
   <input type="color" name="color">

   <p>Website:</p>
   <input type="url" name="website">

</form>
```

Show Image Of the output of the code



© Copyright Intellipaat. All rights reserved.

Hands-On

© Copyright Intellipaat. All rights reserved.

Hands-On: Build a User Information Form

Task

Create a **User Profile Form** that collects different types of user information.

Your form must demonstrate the input types taught in this section.

Include fields for:

- Personal information
- Gender selection
- Hobbies
- Profile photo
- A range-based value
- Color selection
- Website URL



Expected Output

The browser should display a user profile form containing different types of input controls. The different controls should visibly behave according to their input type.

© Copyright Intellipaat. All rights reserved.

Labels and Form Controls

© Copyright Intellipaat. All rights reserved.

Labels and Form Controls

A label identifies a form control and helps users understand what information they should enter or select.

The `<label>` element should be properly associated with its form control.

Using for and id

The `for` attribute of `<label>` should match the `id` of the corresponding **form control.**

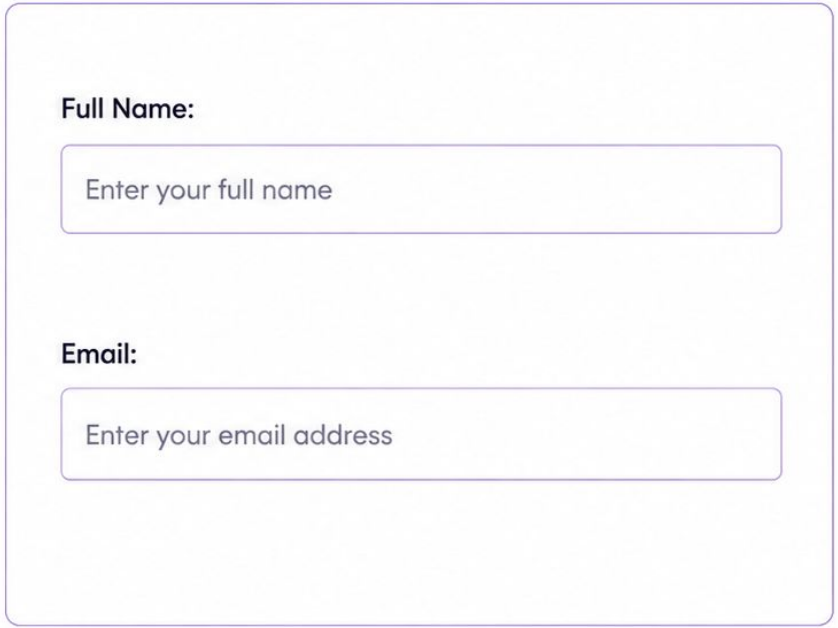
HTML Code

```
<form>
  <label for="name">Full Name:</label>
  <input type="text" id="name" name="name">

  <label for="email">Email:</label>
  <input type="email" id="email" name="email">

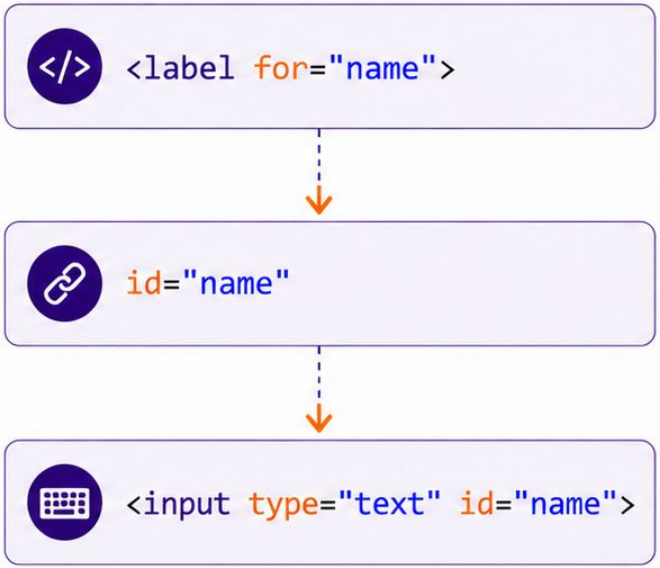
</form>
```

Show Image Of the output of the code



© Copyright Intellipaat. All rights reserved.

How the Association Works



Why Use Labels?

- Clearly identify what information a control requires
- Improve usability
- Allow users to click the label to focus the associated control
- Help assistive technologies understand the form

© Copyright Intellipaat. All rights reserved.

Select Boxes and Text Areas

© Copyright Intellipaat. All rights reserved.

Select Boxes and Text Areas

HTML provides `<select>` and `<textarea>` elements when users need to **choose from options or enter multiple lines of text.**

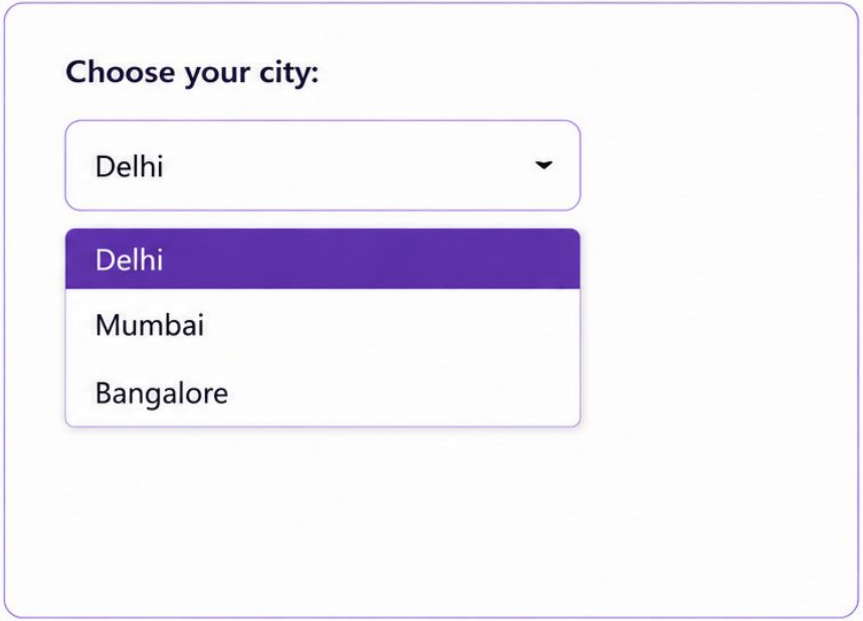
Select Box

The `<select>` element creates a **drop-down list**, while `<option>` defines each available choice.

HTML Code

```
<label for="city">Choose your city:</label>
<select id="city" name="city">
  <option value="delhi">Delhi</option>
  <option value="mumbai">Mumbai</option>
  <option value="bangalore">Bangalore</option>
</select>
```





© Copyright Intellipaat. All rights reserved.

Select Boxes and Text Areas

Text Area

The <textrea> element is used for **multi-line** text input.

HTML Code

```
<label for="message">Message:</label>

<textarea id="message" name="message"
rows="4" cols="40">
</textarea>
```

Output (in Browser)

Message:

write something.....

© Copyright Intellipaat. All rights reserved.

Hands-On

© Copyright Intellipaat. All rights reserved.

Hands-On: Build a Feedback Form

Task

Create a **Website Feedback Form**.

The form should collect:

- Name
- Email
- City
- Feedback message
- Rating or preferred option

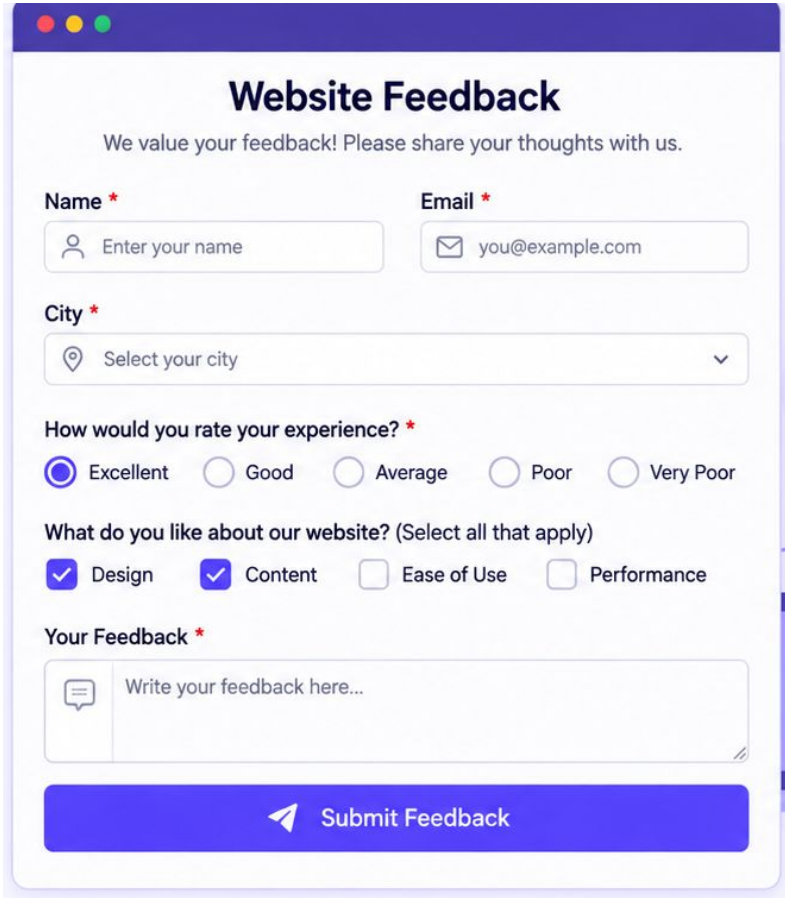
Use appropriate form controls for each type of information.

Expected Output

The browser should display a properly organized feedback form containing:

- Clearly identifiable labels
- A dropdown selection
- Multiple selectable options where appropriate
- A multi-line feedback area

The labels should correctly correspond to their form controls.



© Copyright Intellipaat. All rights reserved.

Buttons

© Copyright Intellipaat. All rights reserved.

HTML Buttons

The `<button>` element creates a **clickable button** that can perform different actions within a form.

Button Types

Type	Purpose	
`submit`	Submits the form	Submit
`reset`	Resets form controls to their initial values	Reset
`button`	A general-purpose button with no default form action	Click Me

© Copyright Intellipaat. All rights reserved.

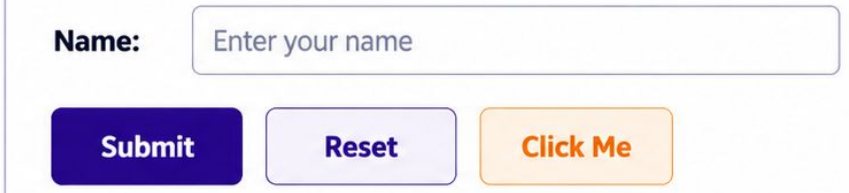
Example

HTML Code

```
<form>
   <label for="name">Name:</label>
   <input type="text" id="name" name="name">

   <button type="submit">Submit</button>
   <button type="reset">Reset</button>
   <button type="button">Click Me</button>
</form>
```

Output



How They Work

- **Submit** → Sends the form data
- **Reset** → Restores initial form values
- **Button** → No default form submission/reset action

Important Point

Always specify the **type** attribute when using `<button>` inside a form so its intended behavior is clear.

© Copyright Intellipaat. All rights reserved.

Hands-On

© Copyright Intellipaat. All rights reserved.

Hands-On: Create a Functional Form Interface

Task

Create a **Form Control Demonstration**.

The form should contain:

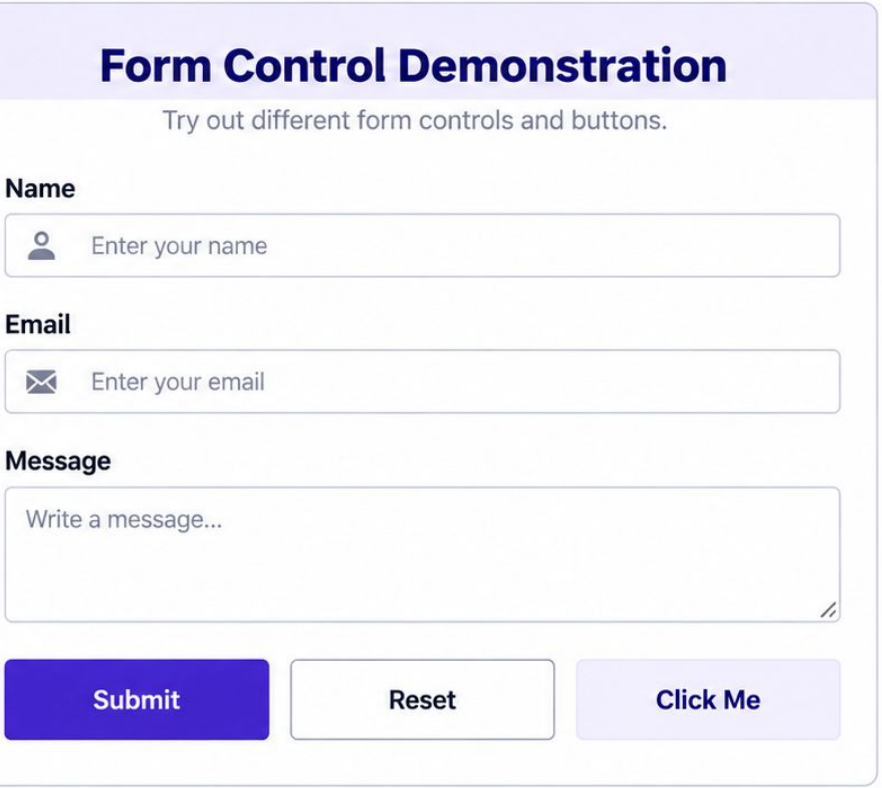
- At least one input field
- A Submit button
- A Reset button
- A general-purpose button

Expected Output

The browser should display all three buttons.

The student should be able to verify:

- Submit performs form submission.
- Reset returns form controls to their initial state.
- The general-purpose button does not perform the default form submission or reset action.



© Copyright Intellipaat. All rights reserved.

Form Attributes

© Copyright Intellipaat. All rights reserved.

Important Form Attributes

Core Idea

Form attributes define how and where the form data is submitted and how the browser handles the form.

Common Form Attributes

Attribute	Purpose	Example
`action`	Specifies where the form data is sent	`action="/register"`
`method`	Specifies how the data is submitted	`method="post"`
`name`	Gives the form a name for identification	`name="registration"`
`autocomplete`	Controls whether the browser can suggest previously entered values	`autocomplete="on"`

© Copyright Intellipaat. All rights reserved.

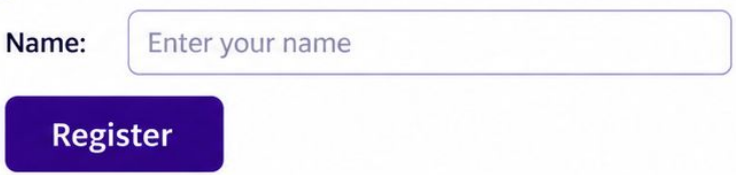
Example

HTML Code

```
<form
  action="/register"
  method="post"
  name="registration"
  autocomplete="on"
 >
  <label for="name">Name:</label>
  <input type="text" id="name" name="name">

  <button type="submit">Register</button>
</form>
```

Output



Understanding the Example

`action="/register"`	→	Where the form data is sent
`method="post"`	→	How the data is submitted
`name="registration"`	→	Identifies the form
`autocomplete="on"`	→	Allows browser suggestions

© Copyright Intellipaat. All rights reserved.

GET vs POST

GET and **POST** are HTTP methods that determine how form data is sent to the server.

GET Method

The **GET** method sends form data as part of the **URL**.

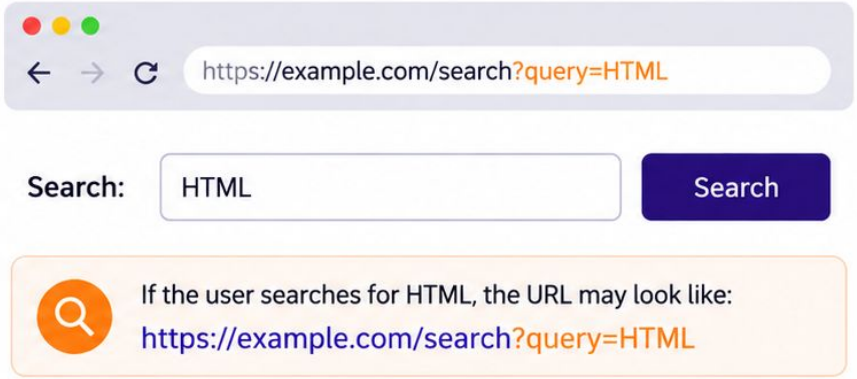
```
<form action="search" method="get">
    <label for="query">Search:</label>
    <input type="text" id="query"
     name="query">

    <button type="submit">Search</button>
</form>
```

Commonly Used For

- Search forms
- Retrieving /fetching information
- Requests that do not change server-side data

Output (Browser View)





© Copyright Intellipaat. All rights reserved.

GET vs POST

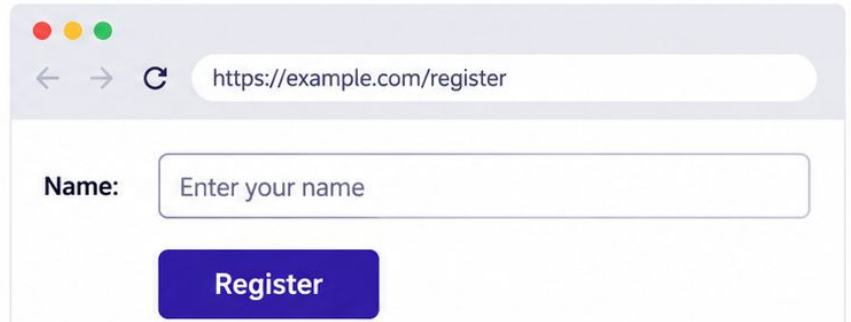
POST Method

The **POST** method sends form data in the **HTTP request body** rather than putting it in the **URL**.

```
<form action="/register" method="post">
    <label for="name">Name:</label>
    <input type="text" id="name" name="name">

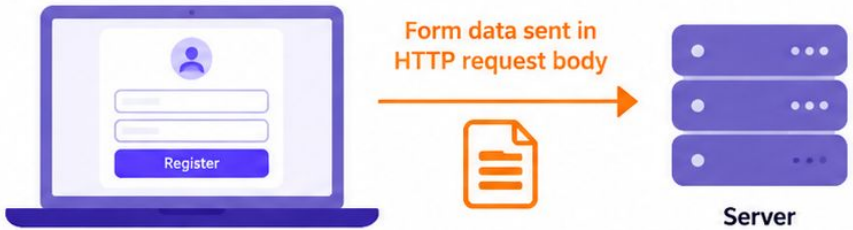
    <button type="submit">Register</button>
</form>
```

Output (Browser View)



Commonly Used For

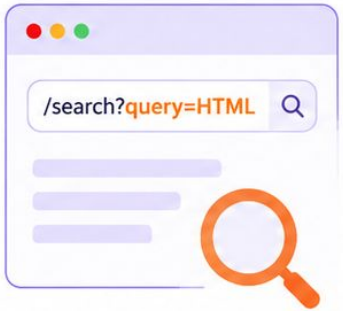
- Registration
- Login
- Submitting information
- Creating or updating server-side data



© Copyright Intellipaat. All rights reserved.

GET vs POST

GET



Aspect	GET	POST
Where data is sent	Data is included in the URL	Data is sent in the request body
Main use	Suitable for retrieving data	Suitable for submitting data
Visibility in URL	Data appears in the URL	Data does not normally appear in the URL
Can be shared	Can be bookmarked/shared as a URL	Not normally represented by a shareable URL
Suitability for sensitive data	Not suitable for sensitive data	Better suited for submitting sensitive data, but **HTTPS is still required**

POST



© Copyright Intellipaat. All rights reserved.

Hands-On

© Copyright Intellipaat. All rights reserved.

Hands-On: Configure Form Submission

Task

Create two small forms:

1. **Form A — Search Form**
   Create a form intended for retrieving/searching information.

2. **Form B — Registration Form**
   Create a form intended for submitting registration information.

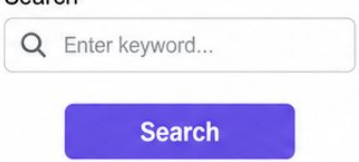
Configure each form according to the submission method appropriate for its purpose.

Expected Output

- The student should be able to demonstrate the difference between the two submission methods through browser behavior.
- For the **search form**, submitted information should be reflected in the URL.
- For the **registration form**, submitted information should be handled through the request body rather than being displayed in the URL.

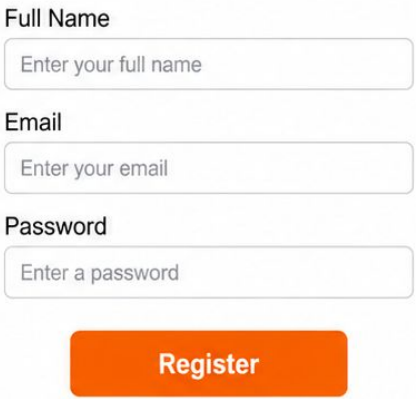
Form A — Search Form

method="GET"



Form B — Registration Form

method="POST"



Note

Observe the browser's address bar and network request to understand how GET and POST work differently.

© Copyright Intellipaat. All rights reserved.

Basic Form Validation

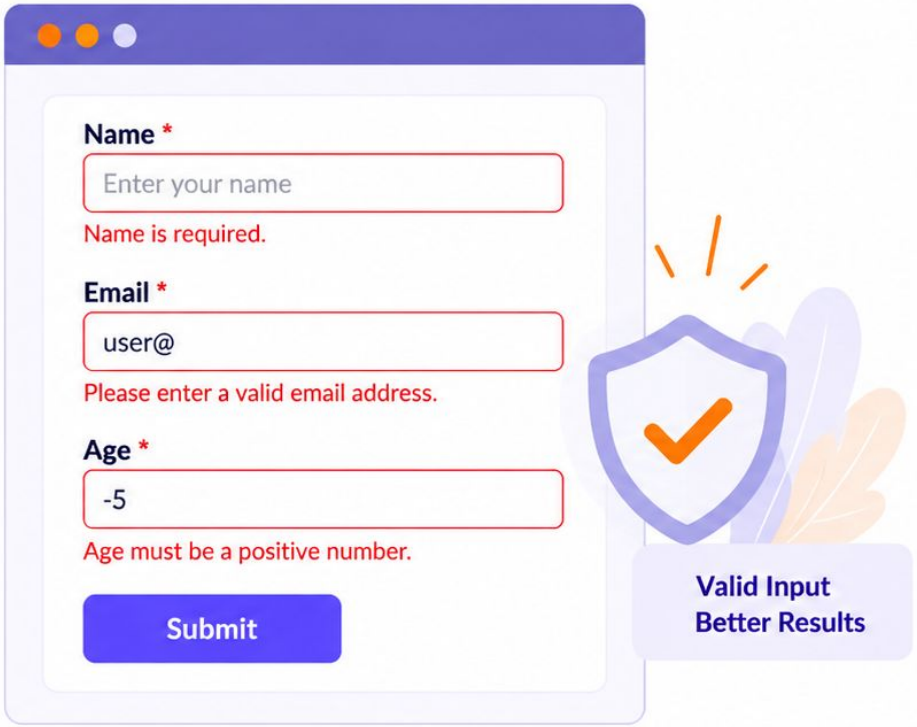
© Copyright Intellipaat. All rights reserved.

HTML Form Validation

Form validation checks whether the data entered by a user meets the required conditions **before the form is submitted.**

Why Is Validation Needed?

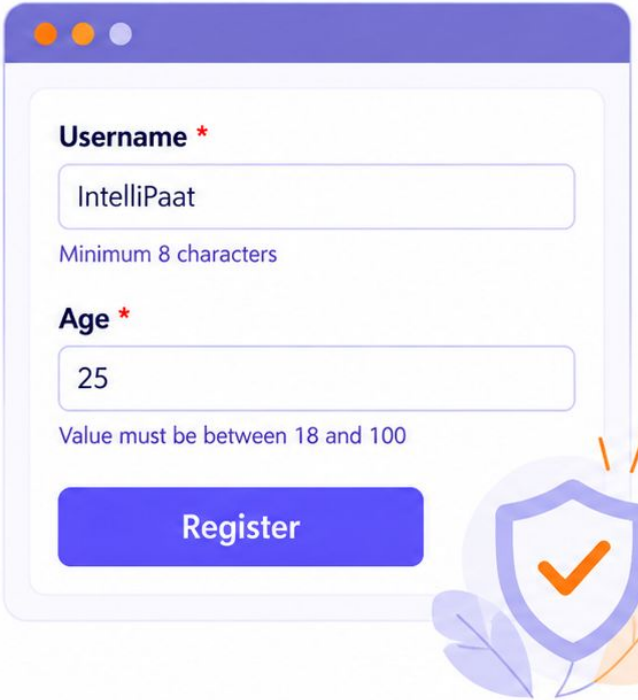
- Prevents incomplete or incorrect data
- Ensures required information is provided
- Improves the quality of submitted data
- Provides immediate feedback to users



© Copyright Intellipaat. All rights reserved.

Common Validation Attributes

Attribute	Purpose	Example
`required`	Makes a field mandatory	`required`
`minlength`	Sets minimum text length	`minlength="8"`
`maxlength`	Sets maximum text length	`maxlength="50"`
`min`	Sets minimum numeric value	`min="18"`
`max`	Sets maximum numeric value	`max="100"`



These validation attributes help ensure that users enter the **correct data** in a form before it is submitted.

© Copyright Intellipaat. All rights reserved.

Example

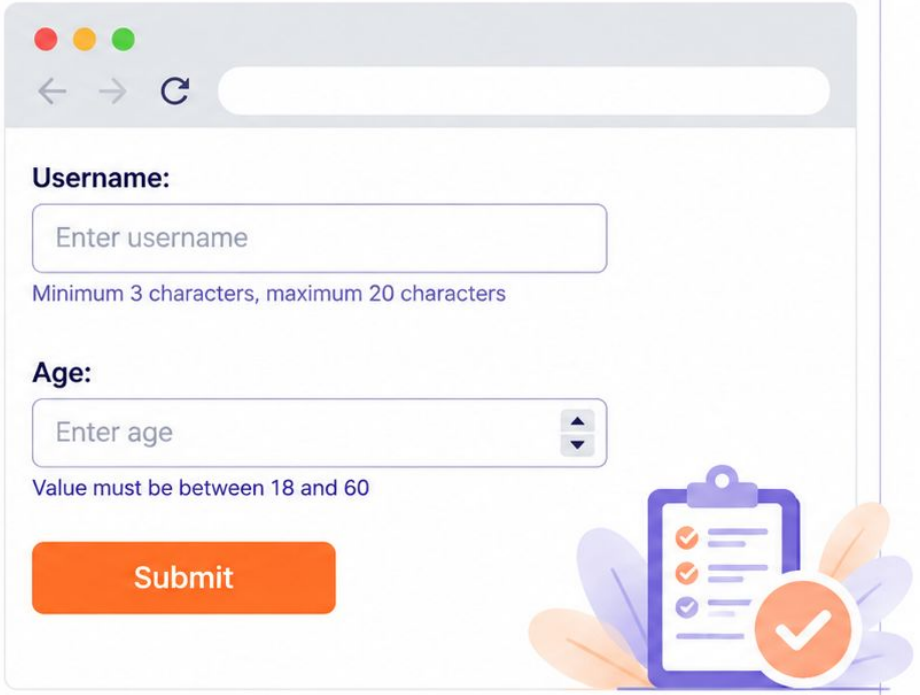
HTML Code

```
<form>
  <label for="username">Username:</label>
  <input type="text"
         id="username"
         name="username"
         required
         minlength="3"
         maxlength="20">

  <label for="age">Age:</label>
  <input type="number"
         id="age"
         name="age"
         min="18"
         max="60">

  <button type="submit">Submit</button>
</form>
```

Output (Browser View)



© Copyright Intellipaat. All rights reserved.

Validation with Input Types

HTML input types and the `pattern` attribute provide **additional ways to control the type and format of user input.**

Input Types for Validation

Some input types automatically provide basic validation:

```
<input type="email" name="email">

<input type="number" name="age">

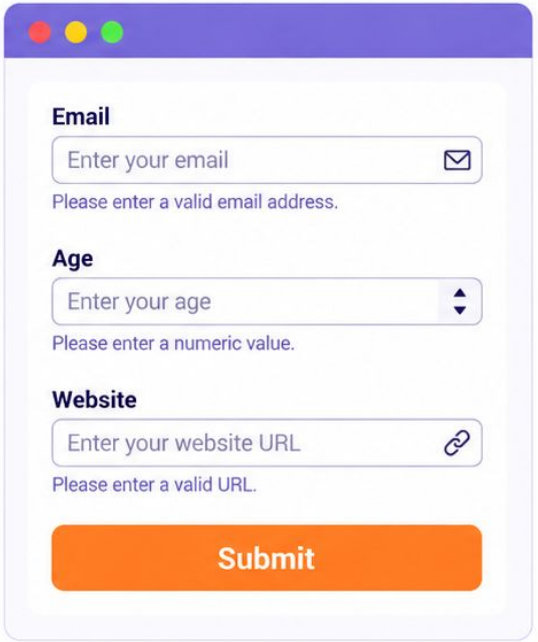
<input type="url" name="website">
```

For example:

email
Checks whether the entered value has a valid email format

number
Expects a numeric value

url
Checks for a valid URL format



Using appropriate input types helps ensure that users enter data in the correct format, making forms more reliable and user-friendly.

© Copyright Intellipaat. All rights reserved.

Validation Using pattern

Using pattern

The `pattern` attribute specifies a **regular expression** that the input value must match.

```
<label for="code">Product Code:</label>
<input
  type="text"
  id="code"
  name="code"
  pattern="[A-Z]{3}[0-9]{3}"
  placeholder="ABC123"
  required>
```

Here:

[A-Z]{3} → Exactly 3 uppercase letters
[0-9]{3} → Exactly 3 digits

Expected format → **ABC123**

Using placeholder

The `placeholder` attribute provides a short hint about the expected input.

```
<input
  type="text"
  placeholder="Enter your product code">
```

Note: A placeholder is a hint, not a replacement for a `<label>`.

Example Input Field

Product Code:

Enter your product code



The **pattern** and **placeholder** attributes help guide users to enter data in the correct format, improving form accuracy and user experience.

© Copyright Intellipaat. All rights reserved.

Hands-On

© Copyright Intellipaat. All rights reserved.

Hands-On: Add Form Validation

Task

Create a **Validated Registration Form.**

The form should contain fields such as:

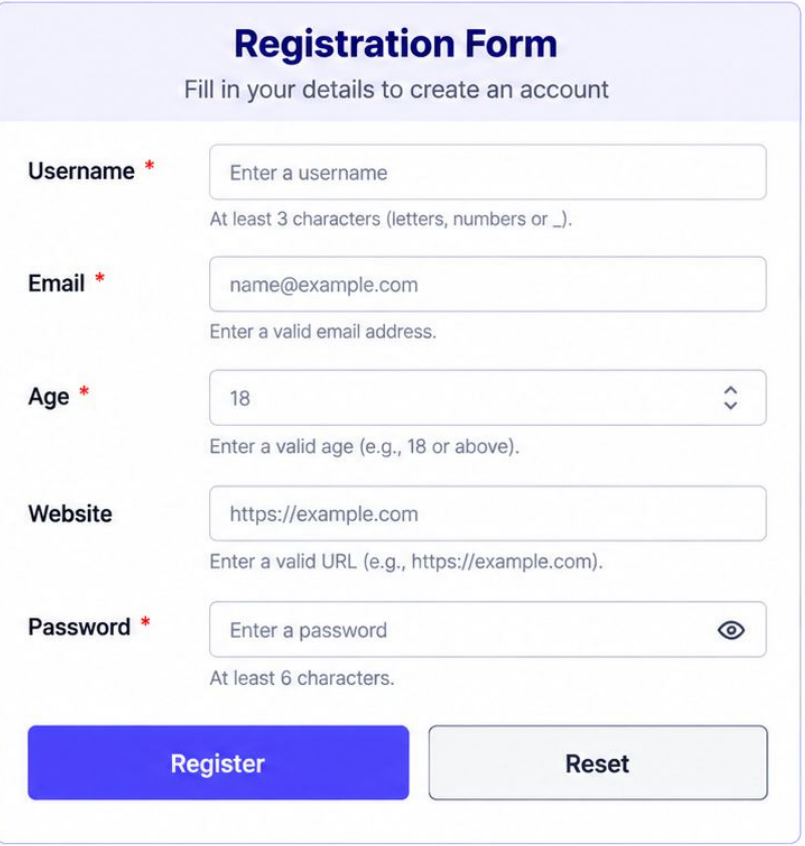
- Username
- Email
- Age
- Website
- Password or another text field

Apply appropriate validation requirements to the fields.

The form should include different validation conditions taught in this section.

Expected Output

- The browser should prevent submission when required information is missing or when entered values do not satisfy the specified validation conditions.
- Valid information should be accepted by the browser.
- The form should also contain meaningful input guidance where appropriate.



© Copyright Intellipaat. All rights reserved.

Semantic HTML Elements

© Copyright Intellipaat. All rights reserved.

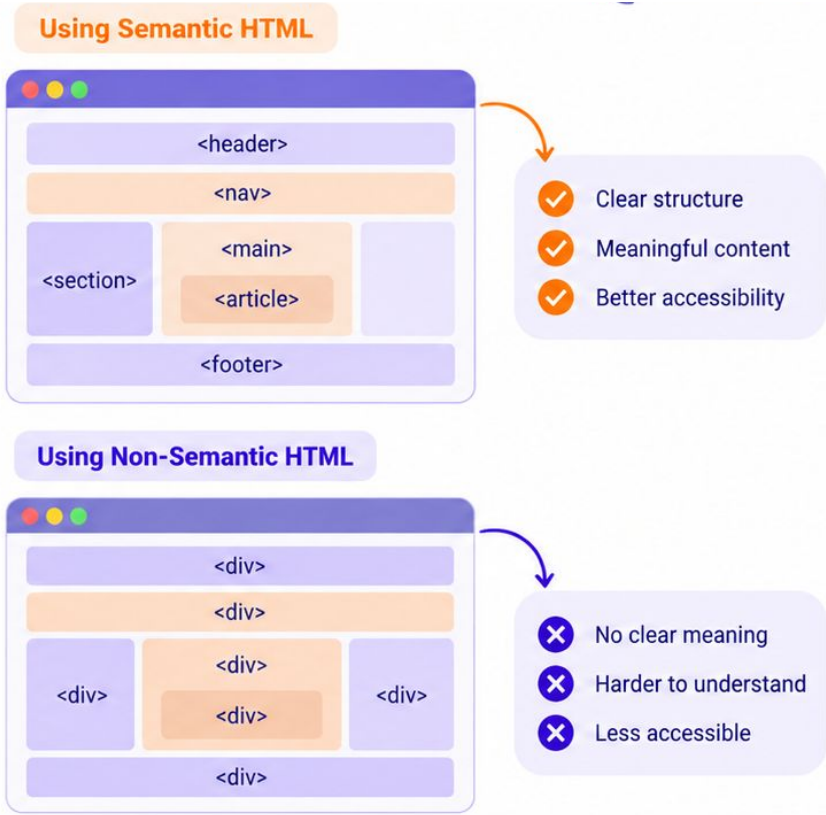
What Is Semantic HTML?

Semantic HTML uses HTML elements that clearly describe the **meaning and purpose of their content**.

Instead of using generic elements for everything, semantic elements help **browsers, developers, and assistive technologies understand the structure** of a webpage.

Semantic vs Non-Semantic Elements

Semantic	Non-Semantic
<header>	
<nav>	
<main>	
<section>	
<article>	
<footer>	

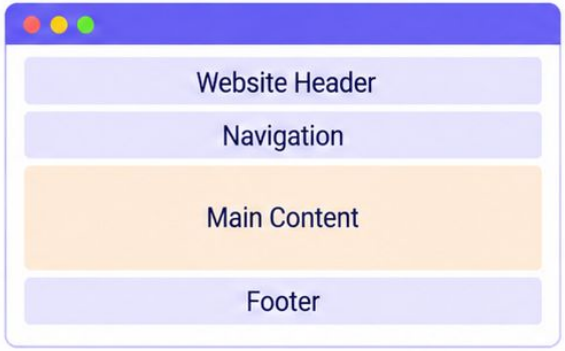


© Copyright Intellipaat. All rights reserved.

Example

Non-semantic structure

```
<div>
   <div>Website Header</div>
   <div>Navigation</div>
   <div>Main Content</div>
   <div>Footer</div>
</div>
```

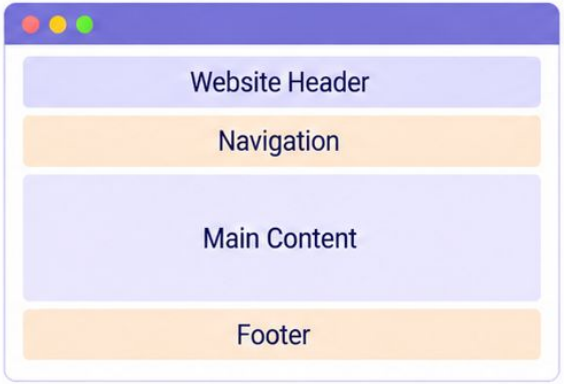


- Uses generic elements

  Does not clearly indicate the purpose of each section

Semantic structure

```
<header>Website Header</header>
<nav>Navigation</nav>
<main>
   Main Content
</main>
<footer>Footer</footer>
```



- Uses semantic HTML elements
- Clearly indicates the purpose of each section
- Improves structure, accessibility, and readability

The semantic structure makes the **purpose of each section clear**.

© Copyright Intellipaat. All rights reserved.

Semantic Page Structure

Semantic elements can be combined to create a **clear and meaningful structure** for a webpage.

Typical Page Structure

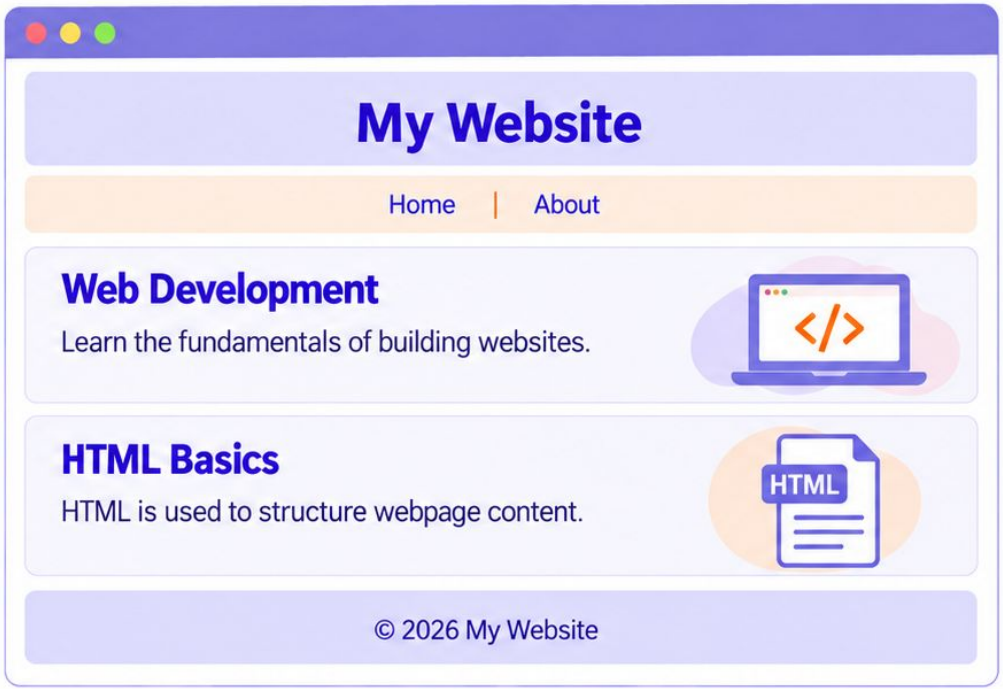
```
<header>
    <h1>My Website</h1>
</header>

<nav>
   <a href="index.html">Home</a>
   <a href="about.html">About</a>
</nav>

<main>
   <section>
      <h2>Web Development</h2>
      <p>Learn the fundamentals of building websites.</p>
   </section>

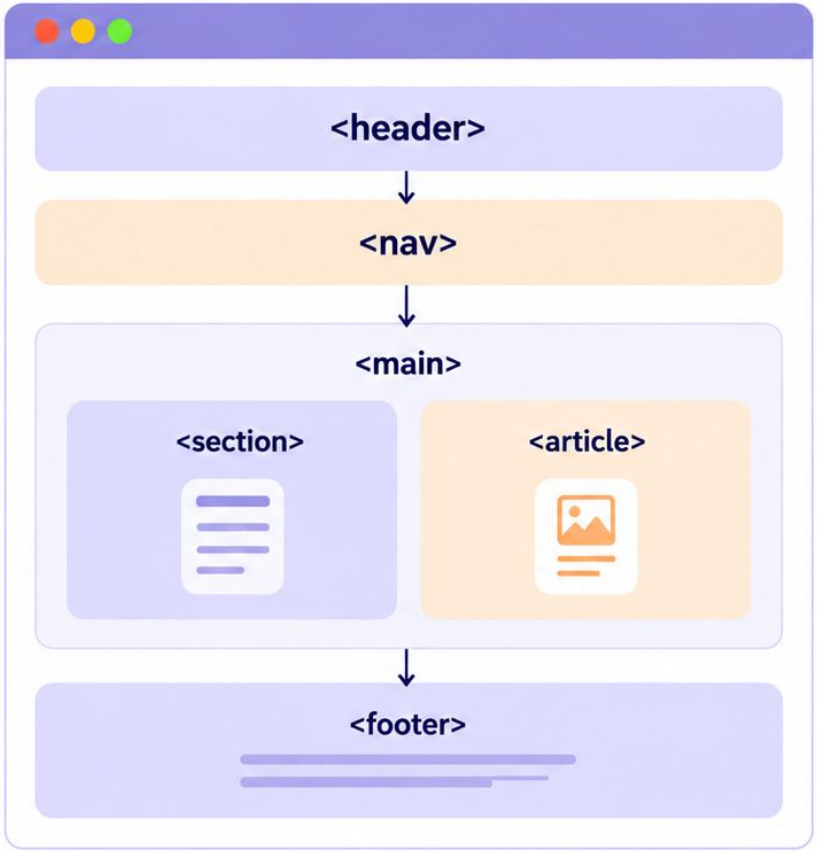
   <article>
      <h2>HTML Basics</h2>
      <p>HTML is used to structure webpage content.</p>
   </article>
</main>

<footer>
   <p>© 2026 My Website</p>
</footer>
```



© Copyright Intellipaat. All rights reserved.

Semantic Page Structure



Why Use This Structure?

- Clearly separates different parts of a webpage
- Makes the document easier to understand
- Helps assistive technologies navigate the page
- Creates a logical content hierarchy

© Copyright Intellipaat. All rights reserved.

Header, Footer, Nav, Main, Section and Article

© Copyright Intellipaat. All rights reserved.

Header, Nav and Main

These semantic elements define the **major structural areas** of a webpage.

<header>

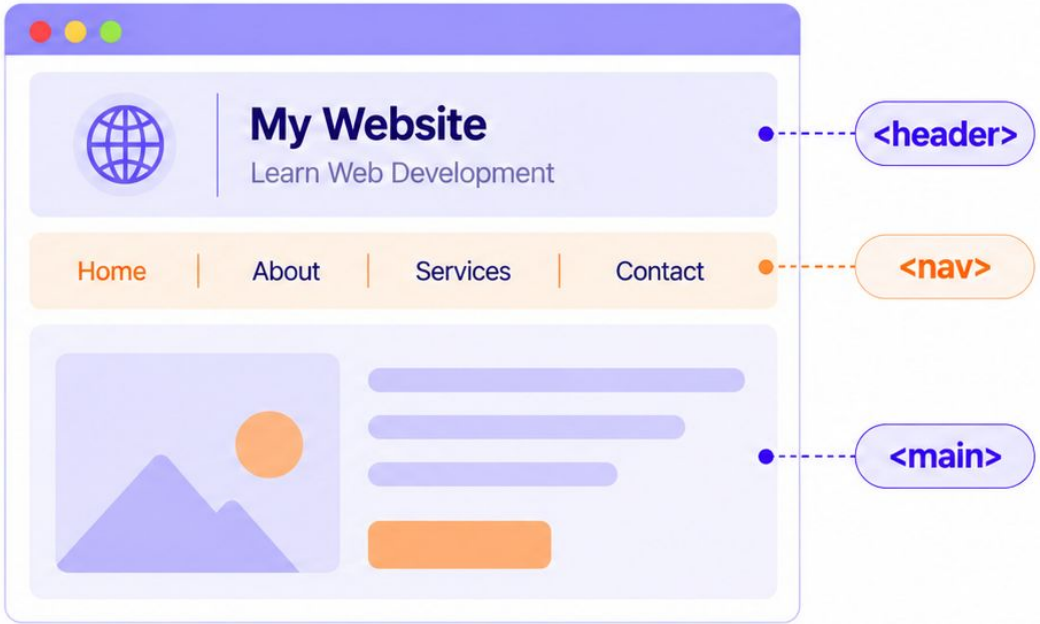
The <header> element represents **introductory content** for a page or section.

It may contain:

- Website or section title
- Logo
- Introductory information

Example Code

```
<header>
  <h1>My Website</h1>
  <p>Learn Web Development</p>
</header>
```



© Copyright Intellipaat. All rights reserved.

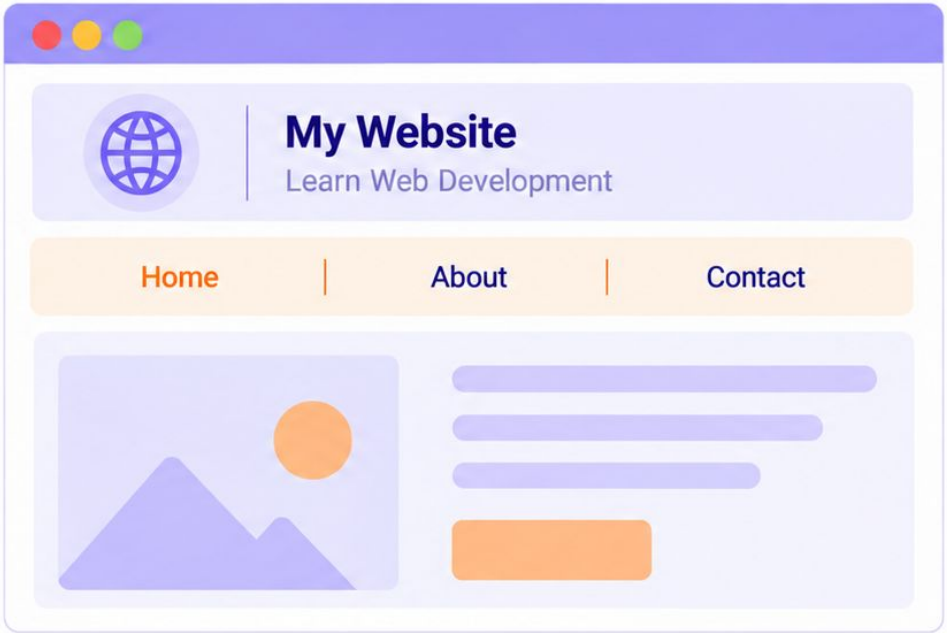
Header, Nav and Main

<nav>

The <nav> element contains **major navigation links**.

Example Code

```
<nav>
  <a href="index.html">Home</a>
  <a href="about.html">About</a>
  <a href="contact.html">Contact</a>
</nav>
```



© Copyright Intellipaat. All rights reserved.

Header, Nav and Main

<main>

The <main> element contains the **primary content** of the webpage.

Example Code

```
<main>
    <h2>Web Development</h2>
    <p>Learn how websites are built.</p>
</main>
```

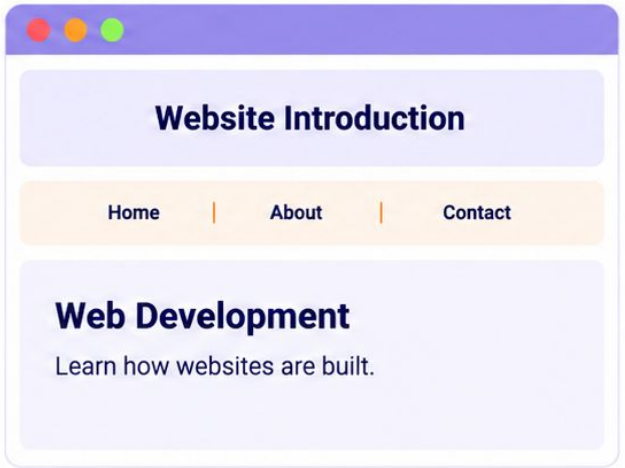
Simple Structure

<header>
Website Introduction

<nav>
Main Navigation

<main>
Primary Page Content

Output of the Code



© Copyright Intellipaat. All rights reserved.

Section, Article and Footer

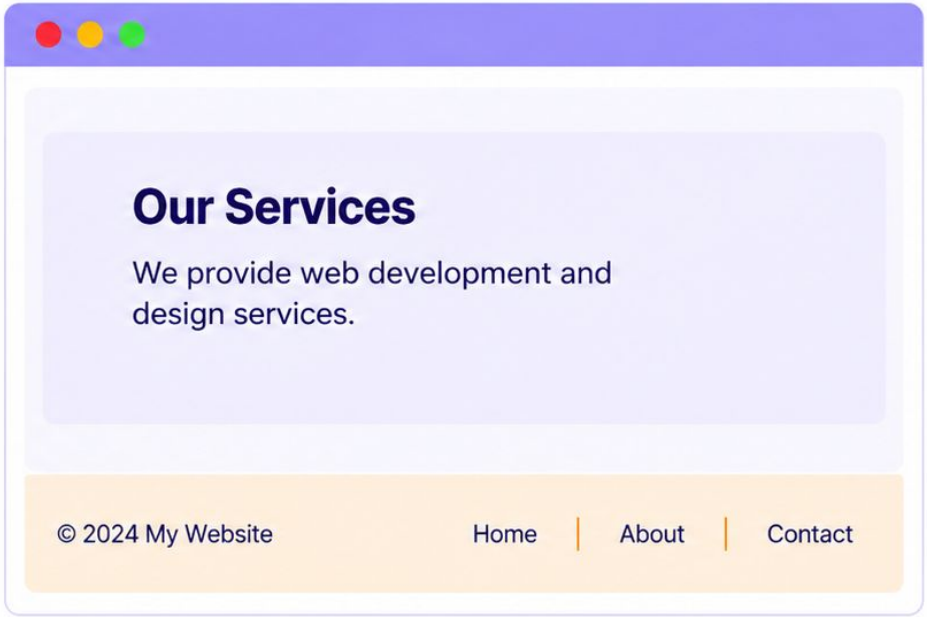
These semantic elements help organize the **main content and supporting information** of a webpage.

<section>

The <section> element represents a **thematic group of related content**, usually with a heading.

Example Code

```
<section>
   <h2>Our Services</h2>
   <p>We provide web development and
      design services.</p>
</section>
```



© Copyright Intellipaat. All rights reserved.

Section, Article and Footer

<article>

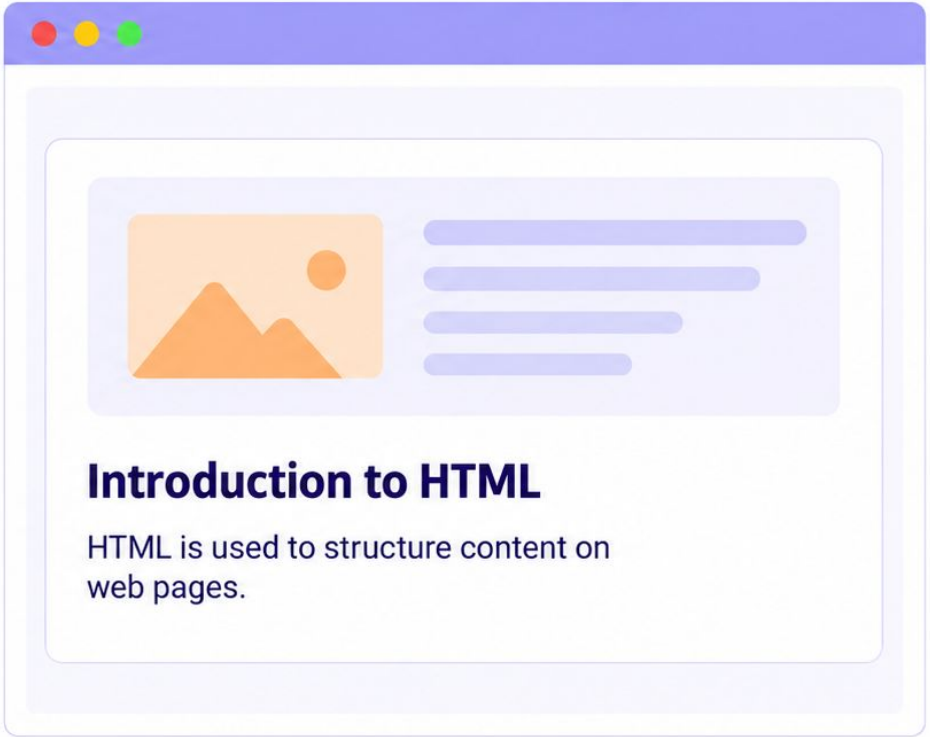
The <article> element represents **self-contained content** that can stand independently.

Common examples include:

- Blog posts
- News articles
- Product reviews

Example Code

```
<article>
  <h2>Introduction to HTML</h2>
  <p>HTML is used to structure content on
     web pages.</p>
</article>
```



© Copyright Intellipaat. All rights reserved.

Section, Article and Footer

<footer>

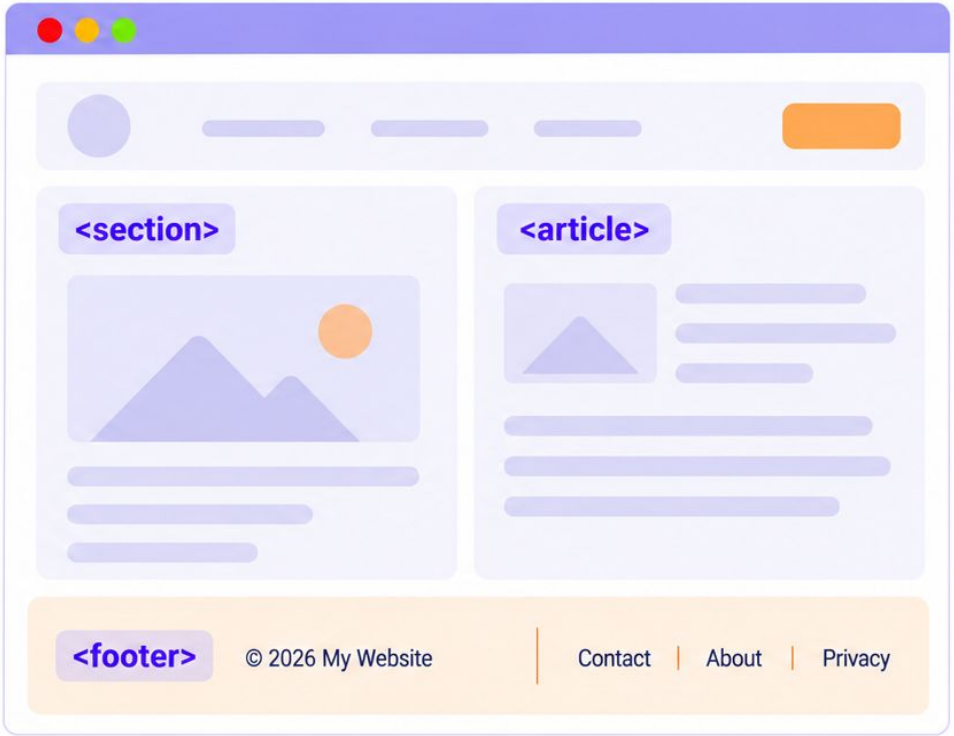
The <footer> element represents footer information for a page or section.

It may contain:

- Copyright information
- Contact information
- Related links

Example Code

```
<footer>
   <p>© 2026 My Website</p>
</footer>
```



© Copyright Intellipaat. All rights reserved.

Hands-On

© Copyright Intellipaat. All rights reserved.

Hands-On: Build a Semantic Webpage

Task

Create a **Student Portfolio Webpage** using semantic HTML.

The webpage should contain:

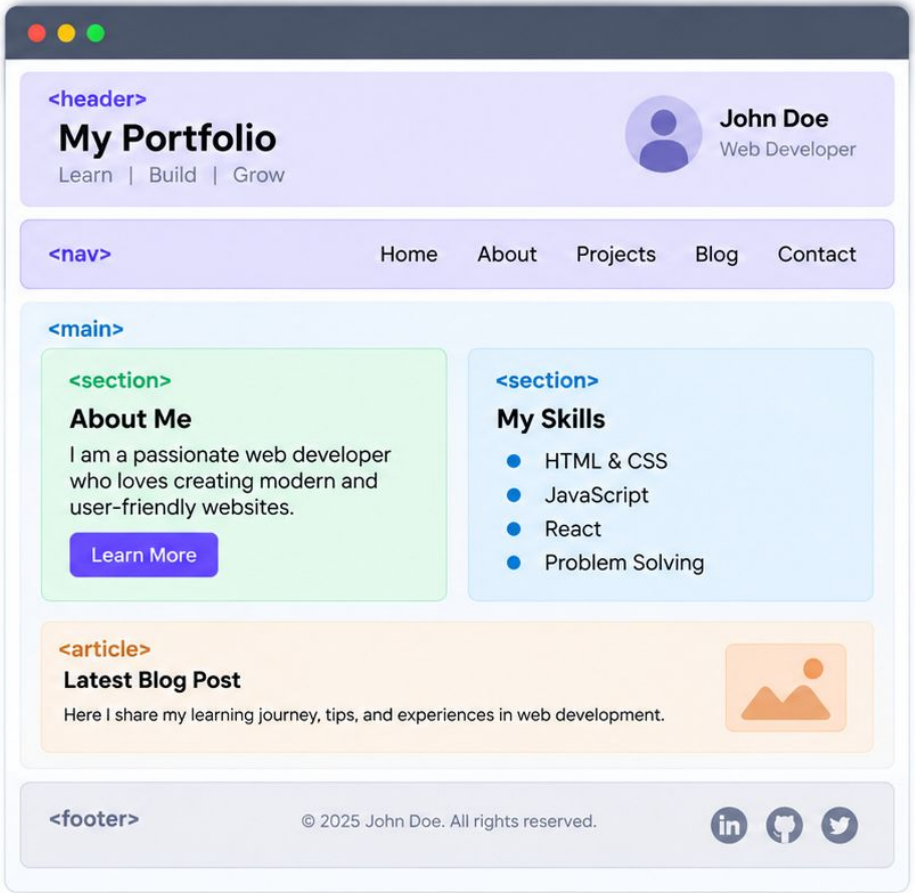
- Website introduction
- Navigation area
- Main content
- At least two related sections
- At least one independent article
- Footer information

Expected Output

The browser should display a complete portfolio-style webpage with clearly separated:

- Header
- Navigation
- Main content
- Sections
- Article content
- Footer

The source code should use semantic HTML to represent the different areas of the webpage.



© Copyright Intellipaat. All rights reserved.

Introduction to Web Accessibility

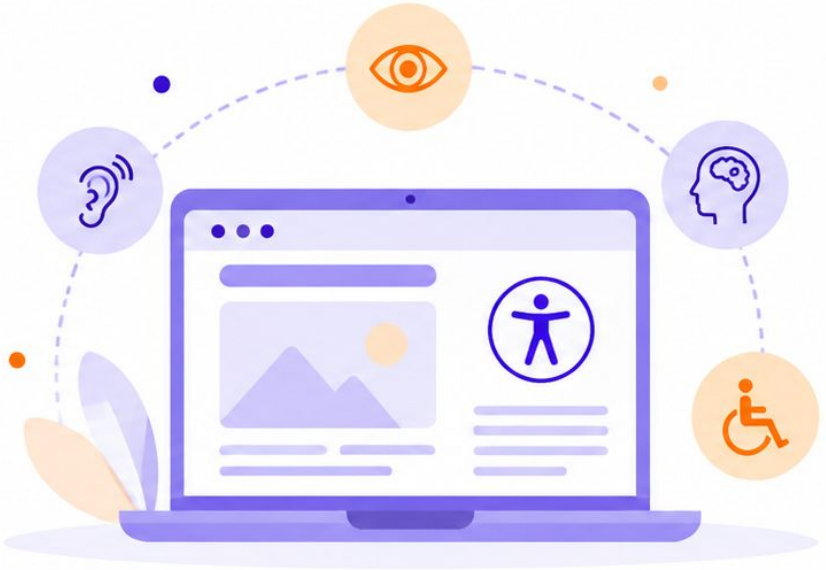
© Copyright Intellipaat. All rights reserved.

What Is Web Accessibility?

Web accessibility means designing websites so that they can be **used and understood by people with different abilities**, including people who use assistive technologies.

Why Is Accessibility Important?

- Allows more people to access and use web content
- Improves usability for everyone
- Supports users who rely on assistive technologies
- Helps create a more inclusive web experience



© Copyright Intellipaat. All rights reserved.

Examples of Accessibility Needs

Examples of Accessibility Needs

Users may:

- Navigate using a **keyboard** instead of a mouse
- Use a **screen reader** to read webpage content
- Have difficulty **seeing** certain visual content
- Have difficulty **hearing** audio content
- Have difficulty using **precise mouse** or touch interactions



© Copyright Intellipaat. All rights reserved.

Accessible HTML

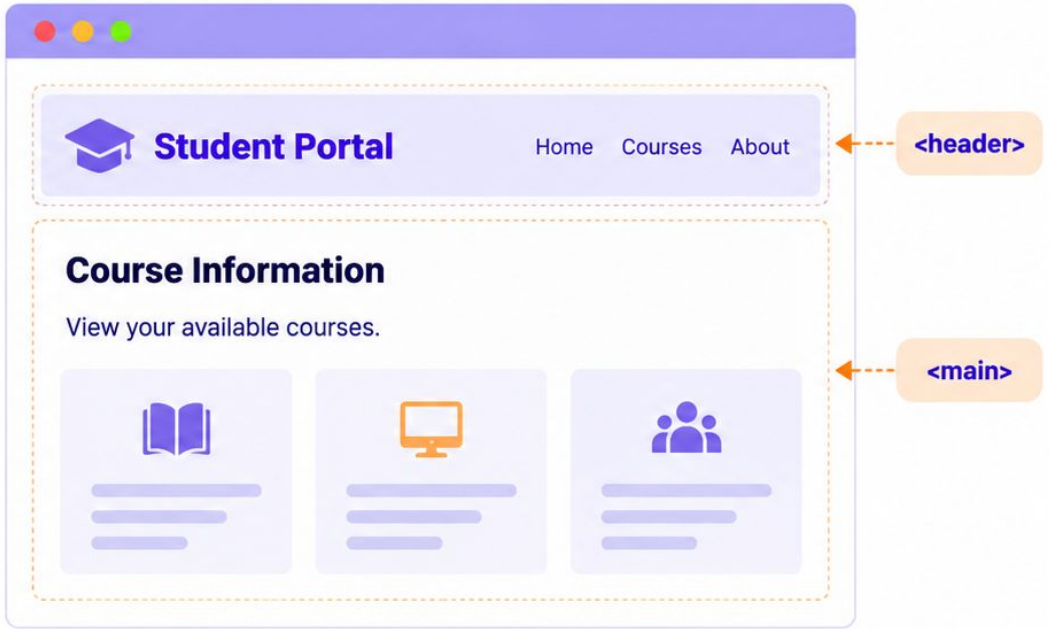
Semantic HTML helps communicate the **meaning and structure** of content to assistive technologies.

Example Code

```
<header>
  <h1>Student Portal</h1>
</header>

<main>
  <h2>Course Information</h2>
  <p>View your available courses.</p>
</main>
```

The meaningful elements help users and assistive technologies understand the page structure.



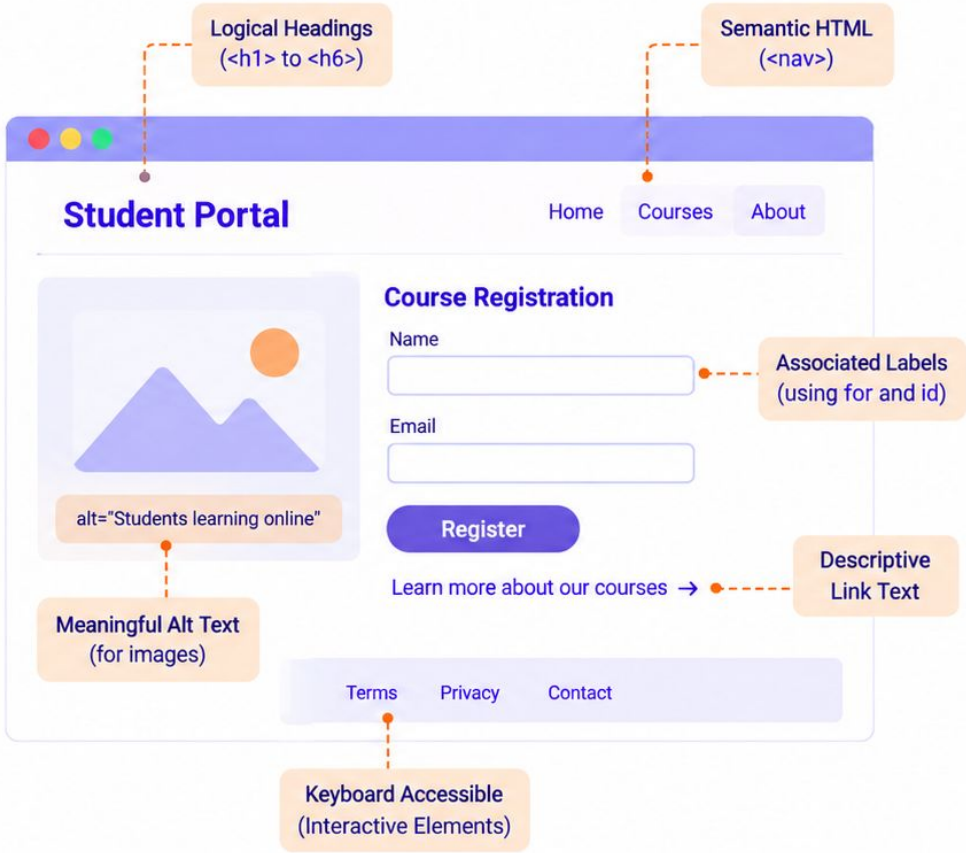
© Copyright Intellipaat. All rights reserved.

Basic Accessibility Practices

Simple HTML practices can make a webpage **more accessible and easier to use.**

Important Practices

1. Use meaningful **alt text** for informative images.
2. Associate labels with form controls using for and **id**.
3. Use a logical heading hierarchy from `<h1>` to `<h6>`.
4. Use semantic HTML such as `<nav>`, `<main>`, `<section>`, and `<article>`.
5. Ensure interactive elements can be accessed using a keyboard.
6. Use descriptive link text that explains the destination or purpose.



© Copyright Intellipaat. All rights reserved.

Examples

Meaningful Image Description

```
<img src="laptop.jpg" alt="Student using a laptop">
```

Accessible Form Label

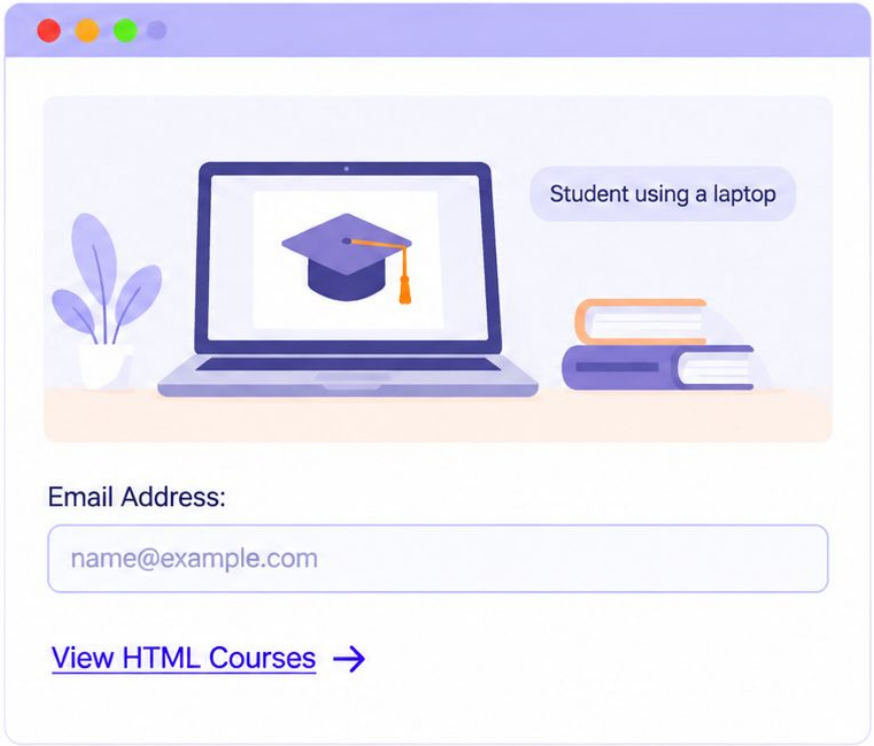
```
<label for="email">Email Address:</label>
<input type="email" id="email" name="email">
```

Descriptive Link

```
<a href="courses.html">View HTML Courses</a>
```

Instead of:

```
<a href="courses.html">Click here</a>
```



© Copyright Intellipaat. All rights reserved.

Hands-On

© Copyright Intellipaat. All rights reserved.

Hands-On: Improve Website Accessibility

Task

Review a previously created webpage and make it more accessible.

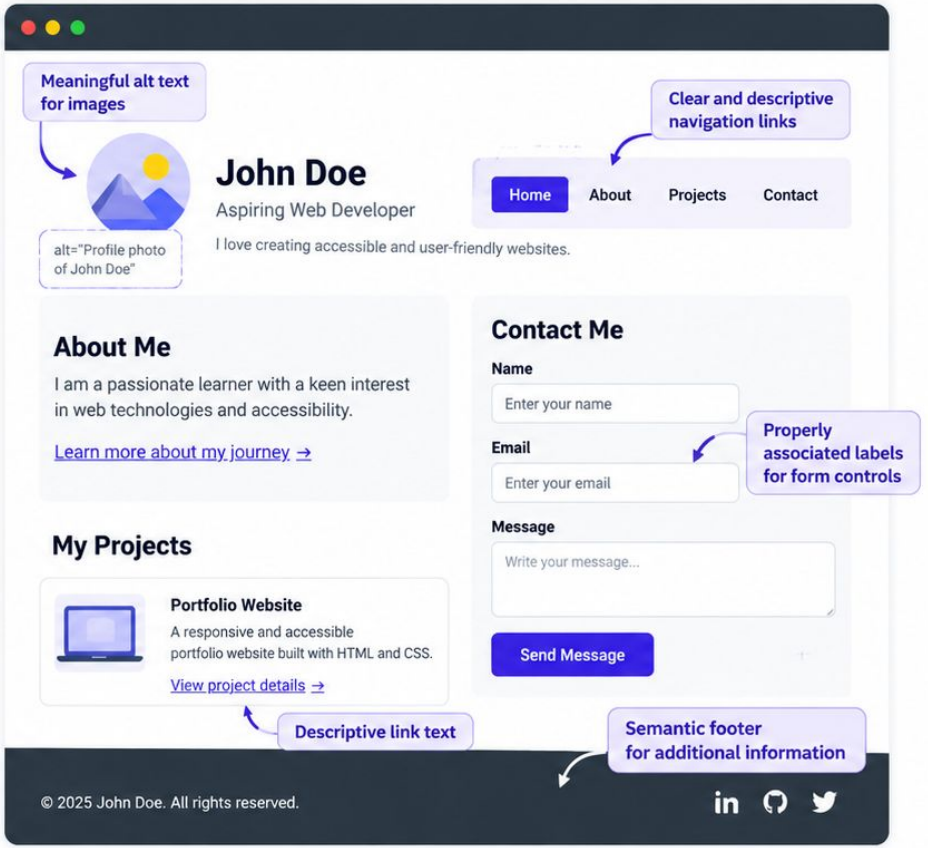
Improve the webpage so that it follows the accessibility practices covered in the module.

The webpage should include:

- Meaningful image descriptions
- Properly associated form labels
- Logical heading hierarchy
- Semantic HTML
- Descriptive link text
- Keyboard-accessible interactive elements

Expected Output

- The revised webpage should be easier to understand and navigate.
- The HTML structure should clearly communicate the purpose of different page sections and interactive elements.



© Copyright Intellipaat. All rights reserved.

HTML Best Practices

© Copyright Intellipaat. All rights reserved.

HTML Best Practices

Following HTML best practices makes code **clean**, **readable**, **accessible**, and **easier to maintain**.

Best Practices

1. Use **semantic HTML elements** whenever appropriate.
2. Maintain **proper indentation** and consistent formatting.
3. Keep elements **properly nested** and closed.
4. Use meaningful `id`, **class**, and `alt` **values**.
5. Use `<label>` elements correctly with form controls.
6. Maintain a **logical heading hierarchy**.
7. Use **descriptive link text**.
8. Avoid **unnecessary HTML elements** and excessive comments.
9. Test webpages in the browser and use **Developer Tools** to identify issues.



© Copyright Intellipaat. All rights reserved.

Example

Poorly organized:

```
<div>
    <div>
        <h1>Courses</h1>
        <p>Learn HTML</p>
    </div>
</div>
```



Better organized:

```
<section>
    <h2>Courses</h2>
    <p>Learn HTML</p>
</section>
```



Good HTML follows **meaningful structure, proper syntax, accessibility practices**, and **consistent organization.**

© Copyright Intellipaat. All rights reserved.

Thanks!



080-4524-9465



support@intellipaat.com

© Copyright Intellipaat. All rights reserved.